APPENDIX

Appendix: Manifests, Schemas, Templates & Full Source Code

Revision: 3 **Last modified:** 2026-09-23T06:50:49Z

> **ARCHITECTURE AMENDMENT — 2026-09-22 (HXC-159 Phase B3).** Appendix section F ("SuperBridge MCP Server — `src/index.ts`") is a FULL TYPESCRIPT SOURCE LISTING for a server that was **never built and does not exist as software** — independently re-derived twice (GitHub `total_count: 0`; npm exact-name lookup `HTTP 404`). Nothing in section F is checked into any git tree; it is a DESIGN artifact, not evidence the server exists (confirmed by the same GitHub/npm sweep). This project's REAL Bridge architecture is 6 layers with no MCP execution tier; the execution role is filled by Spec Kit + Superpowers + Claude Code's own skill/subagent machinery — see `../README.md#architecture-amendment-2026-09-22` for the finding and the worked proof. Section F is retained as historical design intent (§11.4.35/§11.4.124), never a description of shipped software.

Table of Contents

- A. Extension Manifest
- B. Nano-Task Schema
- C. Workable Items Hierarchy Example
- D. MCP Server Configuration
- E. Helix LLM Gateway — `main.go`
- F. SuperBridge MCP Server — `src/index.ts`
- G. Implementation Gate — `implementation-gate.js`
- H. Task Granulator — `granulator.ts`
- I. DAG Engine — `dag.go`
- J. Constitution-Aware Commands — Bash Scripts
- K. Pack Distribution Script — `pack.sh`
- L. Environment Template — `.env.template`
- M. Container Compose Files
- N. Test Artifact Template
- O. CI Configuration — Disabled per §11.4.156

A. Extension Manifest

File: `extension.yml`

```yaml
# extension.yml — SpecKit Superpowers Bridge Extension Manifest
# §11.4.31: helix-deps.yaml declares own-org dependencies

name: speckit-superpowers-bridge
version: "1.0.0"
description: >
  Full SpecKit → Superpowers bridge with nano-task decomposition,
  TDD-enforced implementation gates, DAG-based dependency
        resolution,
  and Helix LLM-integrated autonomous execution.

author: HelixDevelopment
license: Apache-2.0

constitution:
  submodule: constitution
  min_revision: 227
  required_anchors:
    - "11.4.43"   # TDD-fix-discipline
    - "11.4.115"  # RED-baseline + polarity switch
    - "11.4.142"  # Universal code-review
    - "11.4.224"  # TDD for ALL work
    - "11.4.146"  # Reproduce-first-then-extend
    - "11.4.102"  # Systematic-debugging auto-activation
    - "11.4.108"  # Four-layer verification
    - "11.4.69"   # Captured-evidence taxonomy
    - "11.4.161"  # Rootless containers
    - "11.4.156"  # CI disabled

components:
  - name: superbridge-mcp
    type: mcp-server
    language: typescript
    entry: src/index.ts
    runtime: node >=20

        description: MCP server exposing Superpowers skills via
        Model Context Protocol

  - name: implementation-gate
    type: tool
    language: typescript
    entry: src/implementation-gate.ts
    runtime: node >=20
```

```
    description: Mechanical TDD enforcement gate (RED → GREEN →
        REFACTOR)

  - name: task-granulator
    type: tool
    language: typescript
    entry: src/granulator.ts
    runtime: node >=20
    description: Nano-task decomposition engine

  - name: helix-llm-gateway
    type: service
    language: go
    entry: cmd/gateway/main.go
    runtime: go >=1.22
    description: Helix LLM Gateway — routing, auth, context
        budgeting

  - name: dag-engine
    type: library
    language: go
    entry: pkg/dag/dag.go
    runtime: go >=1.22
    description: DAG-based nano-task dependency resolution +
        execution ordering

  - name: constitution-commands
    type: scripts
    language: bash
    entry: scripts/
    description: Constitution-aware CLI helpers

skills:
  - speckit-superpowers-bridge-handoff
  - speckit-superpowers-bridge-execute
  - speckit-superpowers-bridge-guard

mcp_tools:
  - name: tdd_enforce
    description: Enforce TDD (RED → GREEN → REFACTOR) for a nano-
        task
    server: superbridge-mcp
  - name: verify_anti_bluff
    description: Verify a nano-task's TDD verdict is anti-bluff
        compliant
    server: superbridge-mcp
  - name: granulate_task
    description: Decompose a feature task into nano-tasks
```

```
    server: superbridge-mcp
  - name: resolve_dag
    description: Resolve nano-task dependency DAG and return
        execution order
    server: superbridge-mcp
  - name: submit_inference
    description: Submit a nano-task for LLM inference via Helix
        Gateway
    server: superbridge-mcp

directories:
  verdicts: qa-results/tdd_verdicts
  evidence: qa-results/tdd_evidence
  sandboxes: /tmp/tdd_sandboxes
  containers: containers/

dependencies:
  - name: vasic-digital/containers
    type: submodule
    path: submodules/containers
    reason: Container orchestration for nano-task sandboxes
    layout: flat
  - name: vasic-digital/Challenges
    type: submodule
    path: submodules/challenges
    reason: Anti-bluff Challenge banks for TDD enforcement
    layout: flat
  - name: HelixDevelopment/HelixQA
    type: submodule
    path: submodules/helix_qa
    reason: Autonomous QA sessions for bridge validation
    layout: flat
```

B. Nano-Task Schema

File: `schemas/nano-task.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://speckit-bridge.helix/schemas/nano-
      task.schema.json",
  "title": "NanoTask",
  "description": "Schema for a single decoupled nano-task in the
      SpecKit-Superpowers bridge.",
  "type": "object",
  "required": [
    "id",
```

```
    "parent_feature_id",
    "title",
    "description",
    "language",
    "test_file",
    "impl_file",
    "layer",
    "dependencies",
    "acceptance_criteria"
  ],
  "properties": {
    "id": {
      "type": "string",
      "pattern": "^(ATM|SPK)-\\d{3,}-NT-\\d{3,}$",
      "description": "Stable nano-task identifier: <parent>-NT-
        <sequence>"
    },
    "parent_feature_id": {
      "type": "string",
      "pattern": "^(ATM|SPK)-\\d{3,}$",
      "description": "Parent workable item identifier"
    },
    "title": {
      "type": "string",
      "minLength": 10,
      "maxLength": 200,
      "description": "Human-readable nano-task title"
    },
    "description": {
      "type": "string",
      "minLength": 50,
      "description": "Comprehensive description per §11.4.171:
        what, why, how, who, outcome"
    },
    "language": {
      "type": "string",
      "enum": ["typescript", "javascript", "go", "bash", "yaml",
        "json", "markdown"],
      "description": "Implementation language"
    },
    "test_file": {
      "type": "string",
      "description": "Relative path to the test file (must be
        authored FIRST per §11.4.224)"
    },
    "impl_file": {
      "type": "string",
```

```
    "description": "Relative path to the implementation file
      (created AFTER RED phase)"
  },
  "layer": {
    "type": "integer",
    "minimum": 0,
    "description": "Decomposition layer depth (0 = leaf, N = N
      layers of sub-tasks)"
  },
  "dependencies": {
    "type": "array",
    "items": {
      "type": "string",
      "pattern": "^(ATM|SPK)-\\d{3,}-NT-\\d{3,}$"
    },
    "description":
      "Nano-task IDs this task depends on (must be complete
      before this can start)"
  },
  "acceptance_criteria": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["description", "evidence_class"],
      "properties": {
        "description": {
          "type": "string",
          "minLength": 10
        },
        "evidence_class": {
          "type": "string",
          "enum": ["source", "artifact", "runtime",
      "user_visible"],
          "description": "Evidence class required for closure
      per §11.4.226"
        }
      }
    },
    "minItems": 1
  },
  "estimated_tokens": {
    "type": "integer",
    "minimum": 1,
    "maximum": 4096,
    "description": "Estimated prompt token count for LLM
      inference"
  },
  "context_budget": {
```

```
    "type": "integer",
    "maximum": 4096,
    "default": 2048,
    "description":
      "Maximum context tokens for this nano-task's LLM
      inference"
  },
  "timeout_seconds": {
    "type": "integer",
    "default": 300,
    "description": "Hard timeout for RED/GREEN phase execution"
  },
  "max_cycles": {
    "type": "integer",
    "default": 5,
    "description": "Maximum RED→GREEN cycles before loop
      detection fires"
  },
  "sandbox_profile": {
    "type": "string",
    "enum": ["default", "networked", "gpu", "privileged"],
    "default": "default",
    "description": "Container sandbox security profile"
  },
  "red_mode_env": {
    "type": "object",
    "additionalProperties": { "type": "string" },
    "description": "Environment variables to set for RED phase
      execution"
  },
  "green_mode_env": {
    "type": "object",
    "additionalProperties": { "type": "string" },
    "description": "Environment variables to set for GREEN
      phase execution"
  },
  "tags": {
    "type": "array",
    "items": { "type": "string" },
    "description": "Categorization tags"
  },
  "created_at": {
    "type": "string",
    "format": "date-time"
  },
  "created_by": {
    "type": "string",
    "description": "Canonical handle of creator per §11.4.104"
```

```
    },
    "assigned_to": {
      "type": "string",
      "description": "Canonical handle of assignee per §11.4.104"
    }
  }
}
```

C. Workable Items Hierarchy Example

File: `docs/design/hierarchy_example.yml`

```yaml
# Workable Items Hierarchy — N-Layer Decomposition Example
# Demonstrates the SpecKit → Superpowers → Nano-Task
        decomposition chain.

feature:
  id: "ATM-500"
  title: "User authentication with OAuth 2.0"
  type: Feature
  status: In progress
  spec_kit_phase: implement

  # Layer 1: SpecKit tasks
  tasks:
    - id: "ATM-500-T1"
      title: "Implement OAuth token exchange endpoint"
      type: Task
      language: go
      spec: "POST /auth/token accepting authorization_code,
        returning access_token + refresh_token"
      dependencies: []

    - id: "ATM-500-T2"
      title: "Implement token validation middleware"
      type: Task
      language: go
      spec:
        "Middleware that validates Bearer token, injects user
        context"
      dependencies: ["ATM-500-T1"]

    - id: "ATM-500-T3"
      title: "Implement session persistence"
      type: Task
      language: go
      spec: "Store refresh tokens in Redis with TTL"
```

```yaml
    dependencies: ["ATM-500-T1"]

# Layer 2: Superpowers-decomposed nano-tasks for T1
nano_tasks:
  - id: "ATM-500-T1-NT-001"
    parent: "ATM-500-T1"
    title: "Define OAuth token exchange request/response types"
    layer: 2
    language: go
    test_file: "pkg/auth/token_types_test.go"
    impl_file: "pkg/auth/token_types.go"
    dependencies: []
    acceptance_criteria:
      - description: "TokenRequest struct parses valid JSON"
        evidence_class: source
      - description: "TokenResponse struct serializes to valid
      JSON"
        evidence_class: source

  - id: "ATM-500-T1-NT-002"
    parent: "ATM-500-T1"
    title: "Implement authorization_code → token exchange
      logic"
    layer: 2
    language: go
    test_file: "pkg/auth/token_exchange_test.go"
    impl_file: "pkg/auth/token_exchange.go"
    dependencies: ["ATM-500-T1-NT-001"]
    acceptance_criteria:
      - description: "Valid authorization_code returns
      access_token"
        evidence_class: runtime
      - description: "Expired authorization_code returns 401"
        evidence_class: runtime
      - description: "Invalid grant_type returns 400"
        evidence_class: runtime

  - id: "ATM-500-T1-NT-003"
    parent: "ATM-500-T1"
    title: "Implement JWT access token signing"
    layer: 2
    language: go
    test_file: "pkg/auth/jwt_signing_test.go"
    impl_file: "pkg/auth/jwt_signing.go"
    dependencies: ["ATM-500-T1-NT-001"]
    acceptance_criteria:
      - description: "Generated JWT contains sub, exp, iat
      claims"
```

```yaml
      evidence_class: source
    - description: "JWT signature verifies with known public
    key"
      evidence_class: runtime

- id: "ATM-500-T1-NT-004"
  parent: "ATM-500-T1"
  title: "Wire token exchange endpoint into HTTP router"
  layer: 2
  language: go
  test_file: "pkg/auth/router_wire_test.go"
  impl_file: "pkg/auth/router_wire.go"
  dependencies: ["ATM-500-T1-NT-002", "ATM-500-T1-NT-003"]
  acceptance_criteria:
    - description: "POST /auth/token routes to exchange
    handler"
      evidence_class: runtime
    - description: "Unknown routes return 404"
      evidence_class: runtime

# Visualization of the dependency DAG for T1:
dag_ascii: |
  ATM-500-T1-NT-001 (types)
        ├──────────────┬──────────────┐
        ▼              ▼              │
  ATM-500-T1-NT-002  ATM-500-T1-NT-003
  (exchange logic)   (JWT signing)    │
        │              │              │
        └──────┬───────┘              │
               ▼                      │
        ATM-500-T1-NT-004             │
        (router wiring) ◄─────────────┘
```

D. MCP Server Configuration

> **HISTORICAL — never built (see the top-of-document 2026-09-22 Phase B3 amendment banner).** The `superbridge` entry below configures a server that does not exist as software (independently re-derived twice: GitHub `total_count: 0`; npm exact-name lookup HTTP 404) — a design-intent example, not an operational `mcp.json`. Do not paste it into a live MCP config; `cwd`/`dist/index.js` point at nothing. `codegraph` below it IS real (§11.4.78) and is unaffected by this note.

File: `mcp.json`

```
{
  "mcpServers": {
    "superbridge": {
      "command": "node",
      "args": [
        "dist/index.js"
      ],
      "cwd": "${PROJECT_ROOT}/superbridge-mcp",
      "env": {
        "HELIX_GATEWAY_URL": "https://localhost:3101",
        "TDD_VERDICT_DIR": "qa-results/tdd_verdicts",
        "TDD_EVIDENCE_DIR": "qa-results/tdd_evidence",
        "TDD_ITERATIONS": "3",
        "TDD_COVERAGE_THRESHOLD": "0.85",
        "TDD_MAX_CYCLES": "5",
        "TDD_SANDBOX_ENABLED": "true",
        "LLAMA_RPC_HOST": "localhost",
        "LLAMA_RPC_PORT": "8080",
        "LLAMA_RPC_TLS_ENABLED": "true",
        "GATEWAY_CLIENT_CERT": "${PROJECT_ROOT}/certs/gateway-
        client.crt",
        "GATEWAY_CLIENT_KEY": "${PROJECT_ROOT}/certs/gateway-
        client.key",
        "GATEWAY_CA_CERT": "${PROJECT_ROOT}/certs/ca.crt",
        "RED_MODE": "1"
      },
      "disabled": false
    },
    "codegraph": {
      "command": "npx",
      "args": [
        "@colbymchenry/codegraph",
        "serve",
        "--mcp"
      ],
      "disabled": false
    }
  }
}
```

E. Helix LLM Gateway — `main.go`

File: `cmd/gateway/main.go`

```
package main

import (
```

```
    "context"
    "crypto/sha256"
    "crypto/tls"
    "crypto/x509"
    "encoding/hex"
    "encoding/json"
    "errors"
    "flag"
    "fmt"
    "io"
    "log"
    "net/http"
    "os"
    "os/signal"
    "sync"
    "syscall"
    "time"
)

// ─── Configuration ──────────────────────────────────────────────────────────

type Config struct {
    ListenAddr      string        `json:"listen_addr"`
    MaxTokens       int           `json:"max_tokens"`
    RequestTimeout  time.Duration `json:"request_timeout"`
    Backends        []Backend     `json:"backends"`
    TLSCertFile     string        `json:"tls_cert_file"`
    TLSKeyFile      string        `json:"tls_key_file"`
    TLSClientCAFile string        `json:"tls_client_ca_file"`
    ModelManifest   string        `json:"model_manifest"`
}

type Backend struct {
    Name     string `json:"name"`
    URL      string `json:"url"`
    Priority int
        `json:"priority"` // 1 = native (highest), 2+ = provider
    MaxQueue int    `json:"max_queue"`
}

type ModelManifest struct {
    Models []ModelEntry `json:"models"`
}

type ModelEntry struct {
    Name     string `json:"name"`
    Path     string `json:"path"`
```

```
    SHA256   string `json:"sha256"`
    Source   string `json:"source"`
    Default  bool   `json:"default"`
}

// ─── Gateway ────────────────────────────────────────────────────────

type Gateway struct {
    config     Config
    manifest   ModelManifest
    server     *http.Server
    backendsMu sync.RWMutex
    rateLimits map[string]*RateLimiter
}

type RateLimiter struct {
    lastRequest time.Time
    mu          sync.Mutex
}

// ─── Inference Request/Response ─────────────────────────────────────

type InferenceRequest struct {
    TaskID      string  `json:"task_id"`
    Prompt      string  `json:"prompt"`
    MaxTokens   int     `json:"max_tokens"`
    Temperature float64 `json:"temperature"`
    Model       string  `json:"model,omitempty"`
}

type InferenceResponse struct {
    TaskID       string `json:"task_id"`
    Text         string `json:"text"`
    TokensUsed   int    `json:"tokens_used"`
    Model        string `json:"model"`
    Backend      string `json:"backend"`
    FinishReason string `json:"finish_reason"`
}

// ─── Constructor ────────────────────────────────────────────────────

func NewGateway(cfg Config) (*Gateway, error) {
    gw := &Gateway{
        config:     cfg,
        rateLimits: make(map[string]*RateLimiter),
```

```
    }

    if cfg.ModelManifest != "" {
        data, err := os.ReadFile(cfg.ModelManifest)
        if err != nil {
            return nil, fmt.Errorf("read model manifest: %w",
        err)
        }
        if err := json.Unmarshal(data, &gw.manifest); err != nil
        {
            return nil, fmt.Errorf("parse model manifest: %w",
        err)
        }
    }

    return gw, nil
}

// ─── Model Integrity Verification
        ─────────────────────────────────────────────

func (g *Gateway) verifyModelIntegrity(modelPath, expectedHash
        string) error {
    f, err := os.Open(modelPath)
    if err != nil {
        return fmt.Errorf("open model: %w", err)
    }
    defer f.Close()

    h := sha256.New()
    if _, err := io.Copy(h, f); err != nil {
        return fmt.Errorf("hash model: %w", err)
    }
    actual := hex.EncodeToString(h.Sum(nil))

    if actual != expectedHash {
        return fmt.Errorf("MODEL_INTEGRITY_FAIL: %s hash=%s
        expected=%s",
            modelPath, actual, expectedHash)
    }
    return nil
}

// ─── Context Budget Enforcement
        ──────────────────────────────────────────────

func (g *Gateway) enforceBudget(req InferenceRequest) error {
    if req.MaxTokens <= 0 {
```

```
        req.MaxTokens = g.config.MaxTokens
    }
    if req.MaxTokens > g.config.MaxTokens {
        return fmt.Errorf("REJECTED: requested %d tokens exceeds
        budget %d",
            req.MaxTokens, g.config.MaxTokens)
    }
    return nil
}

// ── Prompt Sanitization
        ──────────────────────────────────────────────────

var credentialPatterns = []string{
    `api[_-]?key`,
    `apikey`,
    `access[_-]?token`,
    `secret`,
    `password`,
    `passwd`,
}

func sanitizePrompt(prompt string) (string, bool) {
    sanitized := prompt
    redacted := false
    for _, pattern := range credentialPatterns {
        // In production, use a proper regexp-based scanner
        if containsCredentialPattern(sanitized, pattern) {
            sanitized = redactPattern(sanitized, pattern)
            redacted = true
        }
    }
    return sanitized, redacted
}

func containsCredentialPattern(s, pattern string) bool {
    return len(s) > 0 && len(pattern) > 0 // Stub — real impl
        uses regexp
}

func redactPattern(s, pattern string) string {
    return s // Stub — real impl redacts matched text
}

// ── Backend Selection (§11.4.196 native-first)
        ──────────────────────────────

func (g *Gateway) selectBackend() (*Backend, error) {
```

```
    g.backendsMu.RLock()
    defer g.backendsMu.RUnlock()

    // Native-first: scan all native backends FIRST
        (§11.4.196(A))
    var bestBackend *Backend
    for i := range g.config.Backends {
        b := &g.config.Backends[i]
        if b.Priority == 1 { // Native
            if g.isOperational(b) {
                bestBackend = b
                break
            }
        }
    }

    // Fallback to providers if no native is operational
    if bestBackend == nil {
        for i := range g.config.Backends {
            b := &g.config.Backends[i]
            if b.Priority > 1 && g.isOperational(b) {
                bestBackend = b
                break
            }
        }
    }

    if bestBackend == nil {
        return nil, errors.New("no operational backend available
        — all are rate-limited or unreachable")
    }

    return bestBackend, nil
}

func (g *Gateway) isOperational(b *Backend) bool {
    rl, ok := g.rateLimits[b.Name]
    if !ok {
        return true
    }
    rl.mu.Lock()
    defer rl.mu.Unlock()

    // Rate-limited if last request was within cooldown window
    cooldown := 5 * time.Second
    return time.Since(rl.lastRequest) > cooldown
}
```

```go
// ── Inference Handler ──────────────────────────────────────────

func (g *Gateway) handleInference(w http.ResponseWriter, r
        *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed",
        http.StatusMethodNotAllowed)
        return
    }

    body, err := io.ReadAll(io.LimitReader(r.Body,
        1<<20)) // 1MB limit
    if err != nil {
        http.Error(w, "Failed to read request body",
        http.StatusBadRequest)
        return
    }
    defer r.Body.Close()

    var req InferenceRequest
    if err := json.Unmarshal(body, &req); err != nil {
        writeJSON(w, http.StatusBadRequest, map[string]string{
            "error": fmt.Sprintf("Invalid JSON: %v", err),
        })
        return
    }

    // Enforce context budget
    if err := g.enforceBudget(req); err != nil {
        writeJSON(w, http.StatusPaymentRequired,
        map[string]string{"error": err.Error()})
        return
    }

    // Sanitize prompt for credentials
    sanitized, redacted := sanitizePrompt(req.Prompt)
    if redacted {
        log.Printf("WARNING: Credential pattern redacted in task
        %s", req.TaskID)
    }
    req.Prompt = sanitized

    // Select backend
    backend, err := g.selectBackend()
    if err != nil {
        writeJSON(w, http.StatusServiceUnavailable,
        map[string]string{"error": err.Error()})
```

```
        return
    }

    // Forward to backend (simplified — real impl calls llama.cpp
        RPC)
    ctx, cancel := context.WithTimeout(r.Context(),
        g.config.RequestTimeout)
    defer cancel()

    resp := g.forwardInference(ctx, backend, req)

    // Log audit event
    g.logAuditEvent(req.TaskID, backend.Name, resp.TokensUsed)

    writeJSON(w, http.StatusOK, resp)
}

func (g *Gateway) forwardInference(ctx context.Context, backend
        *Backend, req InferenceRequest) InferenceResponse {
    // In production: actually forward to llama.cpp RPC or cloud
        provider
    // This is a stub returning a placeholder response
    select {
    case <-ctx.Done():
        return InferenceResponse{
            TaskID:       req.TaskID,
            FinishReason: "timeout",
        }
    default:
        return InferenceResponse{
            TaskID:       req.TaskID,
            Text:         "[stub inference response]",
            TokensUsed:   len(req.Prompt) / 4,
            Model:        "local-model",
            Backend:      backend.Name,
            FinishReason: "stop",
        }
    }
}

// ─── Health Check
        ──────────────────────────────────────────────────────

func (g *Gateway) handleHealth(w http.ResponseWriter, r
        *http.Request) {
    backends := make([]map[string]interface{}, 0,
        len(g.config.Backends))
    for _, b := range g.config.Backends {
```

```
        backends = append(backends, map[string]interface{}{
            "name":        b.Name,
            "operational": g.isOperational(&b),
        })
    }

    writeJSON(w, http.StatusOK, map[string]interface{}{
        "status":   "healthy",
        "backends": backends,
        "budget": map[string]int{
            "max_tokens": g.config.MaxTokens,
        },
    })
}

// ─── Audit Logging ──────────────────────────────────────────────

func (g *Gateway) logAuditEvent(taskID, backend string,
        tokensUsed int) {
    entry := map[string]interface{}{
        "event":       "inference_complete",
        "task_id":     taskID,
        "backend":     backend,
        "tokens_used": tokensUsed,
        "timestamp":   time.Now().UTC().Format(time.RFC3339),
    }
    data, _ := json.Marshal(entry)
    log.Printf("AUDIT: %s", string(data))
}

// ─── Server Startup ─────────────────────────────────────────────

func (g *Gateway) Start() error {
    mux := http.NewServeMux()
    mux.HandleFunc("/v1/inference", g.handleInference)
    mux.HandleFunc("/health", g.handleHealth)

    g.server = &http.Server{
        Addr:         g.config.ListenAddr,
        Handler:      mux,
        ReadTimeout:  10 * time.Second,
        WriteTimeout: 30 * time.Second,
        IdleTimeout:  120 * time.Second,
    }

    if g.config.TLSCertFile != "" && g.config.TLSKeyFile != "" {
```

```
        tlsConfig := &tls.Config{
            MinVersion: tls.VersionTLS13,
        }

        if g.config.TLSClientCAFile != "" {
            caCert, err := os.ReadFile(g.config.TLSClientCAFile)
            if err != nil {
                return fmt.Errorf("read CA cert: %w", err)
            }
            caCertPool := x509.NewCertPool()
            caCertPool.AppendCertsFromPEM(caCert)
            tlsConfig.ClientAuth = tls.RequireAndVerifyClientCert
            tlsConfig.ClientCAs = caCertPool
        }

        g.server.TLSConfig = tlsConfig
        log.Printf("Starting Helix LLM Gateway on %s (mTLS)",
        g.config.ListenAddr)
        return g.server.ListenAndServeTLS(g.config.TLSCertFile,
        g.config.TLSKeyFile)
    }

    log.Printf("Starting Helix LLM Gateway on %s (plain HTTP —
        development only)", g.config.ListenAddr)
    return g.server.ListenAndServe()
}

func (g *Gateway) Shutdown(ctx context.Context) error {
    return g.server.Shutdown(ctx)
}

// ─── Helpers ──────────────────────────────────────────────────────

func writeJSON(w http.ResponseWriter, status int, data
        interface{}) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(data)
}

// ─── Main ─────────────────────────────────────────────────────────

func main() {
    configPath := flag.String("config", "config/gateway.json",
        "Path to gateway config")
    flag.Parse()
```

```
    data, err := os.ReadFile(*configPath)
    if err != nil {
        log.Fatalf("Failed to read config: %v", err)
    }

    var cfg Config
    if err := json.Unmarshal(data, &cfg); err != nil {
        log.Fatalf("Failed to parse config: %v", err)
    }

    gw, err := NewGateway(cfg)
    if err != nil {
        log.Fatalf("Failed to create gateway: %v", err)
    }

    go func() {
        if err := gw.Start(); err != nil && err !=
        http.ErrServerClosed {
            log.Fatalf("Gateway failed: %v", err)
        }
    }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    log.Println("Shutting down gateway...")
    ctx, cancel := context.WithTimeout(context.Background(),
        30*time.Second)
    defer cancel()

    if err := gw.Shutdown(ctx); err != nil {
        log.Fatalf("Gateway forced shutdown: %v", err)
    }
    log.Println("Gateway stopped.")
}
```

F. SuperBridge MCP Server — `src/index.ts`

File: `src/index.ts`

```
#!/usr/bin/env node

/**
 * SuperBridge MCP Server
 *
```

```
 * Exposes SpecKit-Superpowers bridge capabilities via the
 * Model Context Protocol (MCP). Provides TDD enforcement,
 * anti-bluff verification, task granulation, and DAG resolution.
 *
 * Constitutional anchors: §11.4.43, §11.4.115, §11.4.146,
       §11.4.224
 */

import { Server } from '@modelcontextprotocol/sdk/server/
       index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/
       server/stdio.js';
import {
  CallToolRequestSchema,
  ListToolsRequestSchema,
  Tool,
} from '@modelcontextprotocol/sdk/types.js';
import { execSync } from 'child_process';
import { readFileSync, existsSync, statSync, writeFileSync,
       mkdirSync } from 'fs';
import { join, dirname } from 'path';
import { createHash } from 'crypto';

// ─── Tool Definitions
       ──────────────────────────────────────────────────

const TOOLS: Tool[] = [
  {
    name: 'tdd_enforce',
    description: 'Enforce TDD (RED → GREEN → REFACTOR) for a
       nano-task per §11.4.43/§11.4.115',
    inputSchema: {
      type: 'object',
      properties: {
        command: {
          type: 'string',
          enum: ['red', 'green', 'refactor'],
          description: 'TDD phase to enforce'
        },
        item_id: {
          type: 'string',
          description: 'Nano-task identifier (ATM-NNN-NT-NNN)'
        },
        test_files: {
          type: 'array',
          items: { type: 'string' },
          description: 'Paths to test files'
        },
```

```
      impl_files: {
        type: 'array',
        items: { type: 'string' },
        description: 'Paths to implementation files'
      }
    },
    required: ['command', 'item_id', 'test_files']
  }
},
{
  name: 'verify_anti_bluff',
  description: 'Verify a nano-task TDD verdict is anti-bluff
      compliant per §11.4.146(D3)',
  inputSchema: {
    type: 'object',
    properties: {
      item_id: { type: 'string', description: 'Nano-task
      identifier' },
      red_verdict: { type: 'string', description: 'Path to RED
      verdict JSON' },
      green_verdict: { type: 'string', description: 'Path to
      GREEN verdict JSON' }
    },
    required: ['item_id', 'red_verdict', 'green_verdict']
  }
},
{
  name: 'granulate_task',
  description: 'Decompose a feature task into nano-tasks with
      dependency resolution',
  inputSchema: {
    type: 'object',
    properties: {
      parent_id: { type: 'string', description: 'Parent
      workable item identifier' },
      spec: { type: 'string', description: 'Feature
      specification text' },
      language: { type: 'string', description: 'Target
      implementation language' },
      max_layer_depth: { type: 'integer', default: 3,
      description: 'Maximum decomposition depth' }
    },
    required: ['parent_id', 'spec', 'language']
  }
},
{
  name: 'resolve_dag',
```

```
    description: 'Resolve nano-task dependency DAG and return
        topological execution order',
    inputSchema: {
      type: 'object',
      properties: {
        parent_id: { type: 'string', description:
        'Parent feature identifier' },
        nano_tasks: {
          type: 'array',
          items: {
            type: 'object',
            properties: {
              id: { type: 'string' },
              dependencies: { type: 'array', items: { type:
        'string' } }
            },
            required: ['id']
          }
        }
      },
      required: ['parent_id', 'nano_tasks']
    }
  },
  {
    name: 'submit_inference',
    description: 'Submit a nano-task for LLM inference via Helix
        Gateway',
    inputSchema: {
      type: 'object',
      properties: {
        task_id: { type: 'string' },
        prompt: { type: 'string' },
        max_tokens: { type: 'integer', default: 2048 }
      },
      required: ['task_id', 'prompt']
    }
  }
];

// ── Server ──────────────────────────────────────────────────────────

const server = new Server(
  { name: 'superbridge-mcp', version: '1.0.0' },
  { capabilities: { tools: {} } }
);
```

```
// ── Tool Handlers ──────────────────────────────────────────────

server.setRequestHandler(ListToolsRequestSchema, async () =>
        ({ tools: TOOLS }));

server.setRequestHandler(CallToolRequestSchema, async (request)
        => {
  const { name, arguments: args } = request.params;
  const itemId = (args as any).item_id || (args as any).task_id
        || '';
  const evidenceDir = join('qa-results', 'tdd_evidence', itemId);

  try {
    switch (name) {
      case 'tdd_enforce': {
        const { command, test_files, impl_files } = args as any;
        const gateScript = join(process.cwd(),
        'node_modules/.bin/implementation-gate');

        // Pass through to implementation-gate.js
        const files = [...(test_files || []), ...(impl_files ||
        [])];
        const cmd = `node ${gateScript} ${command} ${itemId} $
        {files.join(' ')}`;

        try {
          const output = execSync(cmd, { encoding: 'utf-8',
        stdio: 'pipe' });
          return {
            content: [{ type: 'text', text: `PASS: $
        {command.toUpperCase()} phase for ${itemId}\n${output}
        ` }]
          };
        } catch (err: any) {
          return {
            content: [{ type: 'text', text: `REJECTED: $
        {command.toUpperCase()} phase for ${itemId}\n$
        {err.stdout || ''}\n${err.stderr || ''}` }],
            isError: true
          };
        }
      }

      case 'verify_anti_bluff': {
        const { red_verdict, green_verdict } = args as any;

        if (!existsSync(red_verdict)) {
```

```
  return {
    content: [{ type: 'text', text: `FAIL: RED verdict
not found at ${red_verdict}` }],
    isError: true
  };
}
if (!existsSync(green_verdict)) {
  return {
    content: [{ type: 'text', text: `FAIL: GREEN verdict
not found at ${green_verdict}` }],
    isError: true
  };
}

const red = JSON.parse(readFileSync(red_verdict,
'utf-8'));
const green = JSON.parse(readFileSync(green_verdict,
'utf-8'));

const findings: string[] = [];

if (red.exit_code === 0) findings.push('RED_BLUFF: RED
passed — tautology');
if (green.exit_code !== 0) findings.push(`GREEN_FAILED:
GREEN exited ${green.exit_code}`);
if (red.target_fingerprint === green.target_fingerprint)
{
  findings.push('GREEN_BLUFF: Identical fingerprints —
fix never deployed');
}
if (red.iterations < 3) findings.push('NONDETERMINISTIC:
RED <3 iters');
if (green.iterations < 3)
findings.push('NONDETERMINISTIC: GREEN <3 iters');
if (!green.prior_red_verdict)
findings.push('MISSING_RED_REF');

for (const f of [...(red.evidence_files || []), ...
(green.evidence_files || [])]) {
  if (!existsSync(f)) {
    findings.push(`MISSING_EVIDENCE: ${f}`);
  } else if (statSync(f).size === 0) {
    findings.push(`EMPTY_EVIDENCE: ${f}`);
  }
}

if (findings.length === 0) {
  return {
```

```
      content: [{ type: 'text', text: `PASS: ${itemId} —
    anti-bluff verified (RED→GREEN, distinct fingerprints,
    ≥3 iters, evidence present) [evidence: $
    {evidenceDir}]` }]
      };
    }
    return {
      content: [{ type: 'text', text: `FAIL: ${itemId} — $
    {findings.length} bluff(s):\n${findings.map(f => `  - $
    {f}`).join('\n')}` }],
      isError: true
    };
  }

  case 'granulate_task': {
    const { parent_id, spec } = args as any;

    // Decompose spec into nano-tasks
    const nanoTasks = decomposeSpec(spec, parent_id, (args
    as any).max_layer_depth || 3);
    const dag = buildDAG(nanoTasks);
    const order = topologicalSort(dag);

    const result = {
      parent_id,
      nano_task_count: nanoTasks.length,
      max_layer: Math.max(...nanoTasks.map(n => n.layer)),
      execution_order: order,
      nano_tasks: nanoTasks
    };

    const resultPath = join(evidenceDir, `GRANULATE_$
    {parent_id}.json`);
    mkdirSync(dirname(resultPath), { recursive: true });
    writeFileSync(resultPath, JSON.stringify(result, null,
    2));

    return {
      content: [{ type: 'text', text: `Granulated $
    {parent_id} into ${nanoTasks.length} nano-tasks across $
    {result.max_layer} layers. Execution order: $
    {order.join(' → ')} [evidence: ${resultPath}]` }]
    };
  }

  case 'resolve_dag': {
    const { nano_tasks } = args as any;
    const dag = buildDAG(nano_tasks);
```

```
        const order = topologicalSort(dag);

        return {
          content: [{
            type: 'text',
            text: `DAG resolved for ${(args as any).parent_id}: $
        {nano_tasks.length} nodes, execution order: $
        {order.join(' → ')}`
          }]
        };
      }

      case 'submit_inference': {
        const { prompt, max_tokens } = args as any;
        const gatewayURL = process.env.HELIX_GATEWAY_URL ||
        'https://localhost:3101';

        // Forward to Helix LLM Gateway (in production: real HTTP
        call)
        const response = {
          task_id: itemId,
          text: '[inference result via gateway]',
          tokens_used: Math.floor(prompt.length / 4),
          model: 'local-model',
          backend: 'llama.cpp-native',
          finish_reason: 'stop'
        };

        return {
          content: [{ type: 'text', text: `Inference complete
        for ${itemId}: ${response.tokens_used} tokens, backend=$
        {response.backend}` }]
        };
      }

      default:
        return {
          content: [{ type: 'text', text: `Unknown tool: ${name}
        ` }],
          isError: true
        };
    }
  } catch (err: any) {
    return {
      content: [{ type: 'text', text: `Error: ${err.message}` }],
      isError: true
    };
  }
```

```
});

// ─── DAG Implementation ──────────────────────────────────────────

interface NanoTaskNode {
  id: string;
  dependencies: string[];
  layer: number;
  title: string;
}

interface DAG {
  nodes: Map<string, NanoTaskNode>;
  adjacency: Map<string, Set<string>>;
}

function buildDAG(tasks: NanoTaskNode[]): DAG {
  const dag: DAG = { nodes: new Map(), adjacency: new Map() };

  for (const task of tasks) {
    dag.nodes.set(task.id, task);
    if (!dag.adjacency.has(task.id)) {
      dag.adjacency.set(task.id, new Set());
    }
    for (const dep of (task.dependencies || [])) {
      if (!dag.adjacency.has(dep)) {
        dag.adjacency.set(dep, new Set());
      }
      dag.adjacency.get(dep)!.add(task.id);
    }
  }

  // Cyclic check
  const visiting = new Set<string>();
  const visited = new Set<string>();

  function dfs(node: string) {
    if (visiting.has(node)) throw new Error(`CYCLE_DETECTED: $
        {node}`);
    if (visited.has(node)) return;
    visiting.add(node);
    for (const neighbor of dag.adjacency.get(node) || []) {
      dfs(neighbor);
    }
    visiting.delete(node);
    visited.add(node);
  }
```

```
  for (const node of dag.nodes.keys()) {
    if (!visited.has(node)) dfs(node);
  }

  return dag;
}

function topologicalSort(dag: DAG): string[] {
  const inDegree = new Map<string, number>();
  const queue: string[] = [];
  const result: string[] = [];

  for (const node of dag.nodes.keys()) {
    inDegree.set(node, 0);
  }
  for (const [from, tos] of dag.adjacency) {
    if (!dag.nodes.has(from)) continue;
    for (const to of tos) {
      inDegree.set(to, (inDegree.get(to) || 0) + 1);
    }
  }

  for (const [node, degree] of inDegree) {
    if (dag.nodes.has(node) && degree === 0) {
      queue.push(node);
    }
  }

  while (queue.length > 0) {
    const node = queue.shift()!;
    result.push(node);
    for (const neighbor of dag.adjacency.get(node) || []) {
      const newDegree = (inDegree.get(neighbor) || 1) - 1;
      inDegree.set(neighbor, newDegree);
      if (newDegree === 0 && dag.nodes.has(neighbor)) {
        queue.push(neighbor);
      }
    }
  }

  if (result.length !== dag.nodes.size) {
    throw new Error(`DAG_RESOLVE_FAILED: Only ${result.length}/$
        {dag.nodes.size} nodes sortable — cycle detected`);
  }

  return result;
}
```

```
function decomposeSpec(spec: string, parentId: string, maxDepth:
        number): NanoTaskNode[] {
  // In production: invoke LLM (via Helix Gateway) to decompose
        the spec into nano-tasks.
  // This stub returns a sample decomposition.
  const tasks: NanoTaskNode[] = [
    {
      id: `${parentId}-NT-001`,
      title: 'Define types and interfaces',
      layer: 0,
      dependencies: []
    },
    {
      id: `${parentId}-NT-002`,
      title: 'Implement core logic',
      layer: 0,
      dependencies: [`${parentId}-NT-001`]
    },
    {
      id: `${parentId}-NT-003`,
      title: 'Wire into existing infrastructure',
      layer: 0,
      dependencies: [`${parentId}-NT-002`]
    }
  ];

  return tasks;
}

// ─── Startup
        ──────────────────────────────────────────────────────────────

async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error('SuperBridge MCP Server started');
}

main().catch(err => {
  console.error('FATAL:', err);
  process.exit(1);
});
```

G. Implementation Gate — `implementation-gate.js`

The full source code for `implementation-gate.js` is provided in TDD_INTEGRATION.md §10. It includes the RED, GREEN, and REFACTOR phase gates, anti-bluff verification, and deterministic consistency enforcement. Reference it from there as the canonical implementation.

H. Task Granulator — `granulator.ts`

File: `src/granulator.ts`

```
#!/usr/bin/env node

/**
 * Task Granulator — Nano-Task Decomposition Engine
 *
 * Decomposes a feature specification into a tree of decoupled
       nano-tasks
 * suitable for autonomous implementation by Helix LLM (§11.4.58
       PWU pipeline).
 *
 * Constitutional anchors: §11.4.224 (TDD for ALL work), §11.4.58
       (parallel PWU)
 */

interface GranulatorConfig {
  maxLayerDepth: number;
  maxTasksPerLayer: number;
  minTaskGranularity: number; // minimum lines of code per nano-
       task
  language: string;
  parentId: string;
}

interface NanoTaskSpec {
  id: string;
  parent_feature_id: string;
  title: string;
  description: string;
  language: string;
  test_file: string;
  impl_file: string;
  layer: number;
  dependencies: string[];
```

```
  acceptance_criteria: { description: string; evidence_class:
        string }[];
  estimated_tokens: number;
}

class TaskGranulator {
  private config: GranulatorConfig;
  private taskCounter: Map<number, number> = new Map(); // layer
        → counter

  constructor(config: GranulatorConfig) {
    this.config = config;
  }

  /**
   * Granulate a feature specification into nano-tasks.
   *
   * @param spec - The feature specification text from SpecKit's
        specify phase
   * @returns Array of nano-task specifications with dependency
        ordering
   */
  granulate(spec: string): NanoTaskSpec[] {
    const tasks: NanoTaskSpec[] = [];

    // Phase 1: Parse the spec into logical work units
    const workUnits = this.parseSpec(spec);

    // Phase 2: Decompose each work unit into nano-tasks
    for (const unit of workUnits) {
      const unitTasks = this.decomposeUnit(unit, 0);
      tasks.push(...unitTasks);
    }

    // Phase 3: Compute dependencies between nano-tasks
    this.computeDependencies(tasks);

    // Phase 4: Validate the decomposition
    this.validateDecomposition(tasks);

    // Phase 5: Sort by layer and dependency order
    tasks.sort((a, b) => {
      if (a.layer !== b.layer) return a.layer - b.layer;
      return a.id.localeCompare(b.id);
    });

    return tasks;
  }
```

```
private parseSpec(spec: string): WorkUnit[] {
  // In production: parse the SpecKit spec (user stories,
      acceptance criteria, technical notes)
  // into a structured set of work units.
  //
  // Each work unit is a self-contained piece of work that can
      be decomposed
  // into nano-tasks. Work units are ordered by dependency
      (earlier units
  // produce types/interfaces/foundations that later units
      consume).

  // Stub implementation: split on double-newline paragraphs
  const paragraphs = spec.split(/\n\n+/).filter(p =>
      p.trim().length > 0);
  const units: WorkUnit[] = [];

  for (let i = 0; i < paragraphs.length; i++) {
    units.push({
      id: i,
      description: paragraphs[i].trim(),
      dependencies: i > 0 ? [i - 1] : [],
      estimatedComplexity: paragraphs[i].length / 100
    });
  }

  return units;
}

private decomposeUnit(unit: WorkUnit, parentLayer: number):
      NanoTaskSpec[] {
  const tasks: NanoTaskSpec[] = [];
  const complexity = unit.estimatedComplexity;

  // How many nano-tasks does this unit need?
  const taskCount = Math.max(1, Math.min(
    Math.ceil(complexity / this.config.minTaskGranularity),
    this.config.maxTasksPerLayer
  ));

  const layer = parentLayer;

  for (let i = 0; i < taskCount; i++) {
    if (layer > this.config.maxLayerDepth) break;

    const seq = this.nextSeq(layer);
```

```
    const id = `${this.config.parentId}-NT-$
      {String(layer).padStart(3, '0')}$
      {String(seq).padStart(3, '0')}`;

    tasks.push({
      id,
      parent_feature_id: this.config.parentId,
      title: this.deriveTitle(unit, i, taskCount),
      description: this.deriveDescription(unit, i, taskCount),
      language: this.config.language,
      test_file: this.deriveTestPath(id),
      impl_file: this.deriveImplPath(id),
      layer,
      dependencies: [], // computed in phase 3
      acceptance_criteria: this.deriveAcceptanceCriteria(unit,
      i, taskCount),
      estimated_tokens: Math.ceil(complexity * 200 / taskCount)
    });
  }

  // If the task is still too complex, recurse into deeper
      layers
  if (layer < this.config.maxLayerDepth && complexity >
      this.config.minTaskGranularity * 2) {
    for (let i = 0; i < Math.min(taskCount, 2); i++) {
      const subUnit: WorkUnit = {
        id: unit.id * 100 + i,
        description: `Sub-task of ${unit.description}: part $
      {i + 1}`,
        dependencies: [],
        estimatedComplexity: complexity / taskCount
      };
      const subTasks = this.decomposeUnit(subUnit, layer + 1);

      // Wire dependencies: sub-tasks depend on their parent
      layer's task
      if (tasks.length > 0) {
        const parentTaskId = `${this.config.parentId}-NT-$
      {String(layer).padStart(3, '0')}${String(i +
      1).padStart(3, '0')}`;
        for (const subTask of subTasks) {
          if (!subTask.dependencies.includes(parentTaskId)) {
            subTask.dependencies.push(parentTaskId);
          }
        }
      }

      tasks.push(...subTasks);
```

```
    }
  }

  return tasks;
}

private computeDependencies(tasks: NanoTaskSpec[]): void {
  const byLayer = new Map<number, NanoTaskSpec[]>();
  for (const task of tasks) {
    if (!byLayer.has(task.layer)) byLayer.set(task.layer, []);
    byLayer.get(task.layer)!.push(task);
  }

  // Layer N tasks depend on layer N-1 tasks (foundational
      layers first)
  for (let layer = 1; layer <= this.config.maxLayerDepth;
      layer++) {
    const currentLayer = byLayer.get(layer) || [];
    const prevLayer = byLayer.get(layer - 1) || [];

    for (const task of currentLayer) {
      // Each task in layer N depends on at least one task in
      layer N-1
      if (prevLayer.length > 0) {
        const depIndex = Math.min(
          currentLayer.indexOf(task),
          prevLayer.length - 1
        );
        task.dependencies.push(prevLayer[depIndex].id);
      }
    }
  }

  // Same-layer sequential tasks depend on the previous task
  for (const [, layerTasks] of byLayer) {
    for (let i = 1; i < layerTasks.length; i++) {
      if (!layerTasks[i].dependencies.includes(layerTasks[i -
      1].id)) {
        layerTasks[i].dependencies.push(layerTasks[i - 1].id);
      }
    }
  }
}

private validateDecomposition(tasks: NanoTaskSpec[]): void {
  const ids = new Set(tasks.map(t => t.id));

  // Check all dependency IDs exist
```

```
  for (const task of tasks) {
    for (const dep of task.dependencies) {
      if (!ids.has(dep)) {
        throw new Error(`VALIDATION_FAILED: Task ${task.id}
      depends on ${dep} which does not exist`);
      }
    }
  }

  // Check for self-dependency
  for (const task of tasks) {
    if (task.dependencies.includes(task.id)) {
      throw new Error(`VALIDATION_FAILED: Task ${task.id}
      depends on itself`);
    }
  }

  // Check description meets §11.4.171 minimum (50 chars)
  for (const task of tasks) {
    if (task.description.length < 50) {
      throw new Error(`VALIDATION_FAILED: Task ${task.id}
      description too short (${task.description.length} < 50
      chars per §11.4.171)`);
    }
  }
}

private nextSeq(layer: number): number {
  const current = this.taskCounter.get(layer) || 0;
  this.taskCounter.set(layer, current + 1);
  return current + 1;
}

private deriveTitle(unit: WorkUnit, index: number, total:
      number): string {
  const prefix = unit.description.split('.')[0] ||
      unit.description;
  return `${prefix.substring(0, 50)} (part ${index + 1}/$
      {total})`;
}

private deriveDescription(unit: WorkUnit, index: number,
      total: number): string {
  return `Part ${index + 1} of ${total} for: $
      {unit.description}. ` +

      `This nano-task is a decoupled, independently testable
      unit of work ` +
```

```
      `that contributes to the parent feature ${this.config.parentId}. ` +
      `It must be implemented following TDD: write the test first, observe it fail (RED), ` +
      `then implement until the test passes (GREEN). ` +
      `The task is small enough that a modest local LLM can execute it correctly.`;
  }

  private deriveTestPath(id: string): string {
    return `nano_tasks/${this.config.parentId}/${id}_test.${this.fileExtension()}`;
  }

  private deriveImplPath(id: string): string {
    return `nano_tasks/${this.config.parentId}/${id}.${this.fileExtension()}`;
  }

  private fileExtension(): string {
    switch (this.config.language) {
      case 'go': return 'go';
      case 'typescript': return 'ts';
      case 'javascript': return 'js';
      case 'bash': return 'sh';
      default: return 'ts';
    }
  }

  private deriveAcceptanceCriteria(unit: WorkUnit, index: number, total: number): { description: string; evidence_class: string }[] {
    const criteria: { description: string; evidence_class: string }[] = [];

    criteria.push({
      description: `Nano-task ${index + 1}/${total} implements its specified sub-function correctly`,
      evidence_class: 'source'
    });

    if (total <= 3) {
      criteria.push({
        description:
        `The implementation passes all assertions in the paired test file`,
        evidence_class: 'runtime'
      });
```

```
    }

    return criteria;
  }
}

// ── Types ──────────────────────────────────────────────────────────

interface WorkUnit {
  id: number;
  description: string;
  dependencies: number[];
  estimatedComplexity: number;
}

// ── CLI ────────────────────────────────────────────────────────────

async function main() {
  const args = process.argv.slice(2);
  if (args.length < 2) {
    console.error('Usage: granulator <parent_id> <spec_file> [--
        language=go] [--max-depth=3]');
    process.exit(1);
  }

  const parentId = args[0];
  const specFile = args[1];
  const fs = await import('fs');

  if (!fs.existsSync(specFile)) {
    console.error(`Spec file not found: ${specFile}`);
    process.exit(1);
  }

  const spec = fs.readFileSync(specFile, 'utf-8');

  const config: GranulatorConfig = {
    maxLayerDepth: 3,
    maxTasksPerLayer: 10,
    minTaskGranularity: 5,
    language: 'go',
    parentId
  };

  // Parse optional flags
  for (const arg of args.slice(2)) {
```

```
    if (arg.startsWith('--language=')) config.language =
        arg.split('=')[1];
    if (arg.startsWith('--max-depth=')) config.maxLayerDepth =
        parseInt(arg.split('=')[1], 10);
  }

  const granulator = new TaskGranulator(config);
  const tasks = granulator.granulate(spec);

  console.log(JSON.stringify({
    parent_id: parentId,
    task_count: tasks.length,
    max_layer: Math.max(...tasks.map(t => t.layer)),
    tasks
  }, null, 2));
}

main().catch(err => {
  console.error('FATAL:', err.message);
  process.exit(1);
});
```

I. DAG Engine — dag.go

File: pkg/dag/dag.go

```
package dag

import (
    "errors"
    "fmt"
    "sort"
    "sync"
)

// Node represents a single nano-task in the dependency DAG.
type Node struct {
    ID           string   `json:"id"`
    Title        string   `json:"title"`
    Layer        int      `json:"layer"`
    Dependencies []string `json:"dependencies"`
    Status       Status   `json:"status"`
}

// Status is the execution status of a DAG node.
type Status string
```

```
const (
    StatusPending Status = "pending"
    StatusReady   Status = "ready"
    StatusRunning Status = "running"
    StatusDone    Status = "done"
    StatusFailed  Status = "failed"
    StatusBlocked Status = "blocked"
)

// DAG is a directed acyclic graph of nano-tasks.
type DAG struct {
    mu         sync.RWMutex
    Nodes      map[string]*Node   `json:"nodes"`
    Adjacency  map[string][]string `json:"adjacency"` // node → dependents
    InDegree   map[string]int      `json:"in_degree"`
    ReadyQueue []string            `json:"ready_queue"`
}

// New creates a new DAG from a set of nodes.
func New(nodes []Node) (*DAG, error) {
    d := &DAG{
        Nodes:     make(map[string]*Node),
        Adjacency: make(map[string][]string),
        InDegree:  make(map[string]int),
    }

    for i := range nodes {
        n := nodes[i]
        n.Status = StatusPending
        d.Nodes[n.ID] = &n
        d.InDegree[n.ID] = 0
        if d.Adjacency[n.ID] == nil {
            d.Adjacency[n.ID] = []string{}
        }
    }

    // Build adjacency list and in-degree counts
    for _, n := range d.Nodes {
        for _, depID := range n.Dependencies {
            if _, ok := d.Nodes[depID]; !ok {
                return nil, fmt.Errorf("UNRESOLVED_DEPENDENCY:
        node %s depends on %s which does not exist", n.ID,
        depID)
            }
            d.Adjacency[depID] = append(d.Adjacency[depID], n.ID)
            d.InDegree[n.ID]++
        }
```

```
    }

    // Cycle detection
    if err := d.detectCycle(); err != nil {
        return nil, err
    }

    // Compute initial ready queue (in-degree == 0)
    d.rebuildReadyQueue()

    return d, nil
}

// detectCycle uses DFS to detect cycles in O(V+E).
func (d *DAG) detectCycle() error {
    const (
        white = 0 // unvisited
        gray  = 1 // in current DFS path
        black = 2 // fully explored
    )

    color := make(map[string]int)
    for id := range d.Nodes {
        color[id] = white
    }

    var dfs func(id string) error
    dfs = func(id string) error {
        color[id] = gray
        for _, neighbor := range d.Adjacency[id] {
            switch color[neighbor] {
            case gray:
                return
        fmt.Errorf("CYCLE_DETECTED: back-edge from %s to %s",
        id, neighbor)
            case white:
                if err := dfs(neighbor); err != nil {
                    return err
                }
            }
        }
        color[id] = black
        return nil
    }

    for id := range d.Nodes {
        if color[id] == white {
            if err := dfs(id); err != nil {
```

```
                return err
            }
        }
    }
    return nil
}

// TopologicalSort returns nodes in topological order (Kahn's
        algorithm).
func (d *DAG) TopologicalSort() ([]string, error) {
    d.mu.RLock()
    defer d.mu.RUnlock()

    inDegree := make(map[string]int)
    for id, deg := range d.InDegree {
        inDegree[id] = deg
    }

    queue := []string{}
    for id, deg := range inDegree {
        if deg == 0 {
            queue = append(queue, id)
        }
    }
    sort.Strings(queue) // Deterministic order

    result := []string{}
    for len(queue) > 0 {
        node := queue[0]
        queue = queue[1:]
        result = append(result, node)

        for _, neighbor := range d.Adjacency[node] {
            inDegree[neighbor]--
            if inDegree[neighbor] == 0 {
                queue = append(queue, neighbor)
            }
        }
    }

    if len(result) != len(d.Nodes) {
        return nil, fmt.Errorf("TOPOLOGICAL_SORT_FAILED: only %d/
        %d nodes sortable — cycle or missing node",
            len(result), len(d.Nodes))
    }

    return result, nil
}
```

```go
// GetReady returns the current set of nodes ready for execution
        (all deps satisfied).
func (d *DAG) GetReady() []string {
    d.mu.RLock()
    defer d.mu.RUnlock()

    ready := make([]string, len(d.ReadyQueue))
    copy(ready, d.ReadyQueue)
    return ready
}

// MarkDone marks a node as complete and updates dependent nodes'
        readiness.
func (d *DAG) MarkDone(id string) error {
    d.mu.Lock()
    defer d.mu.Unlock()

    node, ok := d.Nodes[id]
    if !ok {
        return fmt.Errorf("NODE_NOT_FOUND: %s", id)
    }

    if node.Status == StatusDone {
        return nil // Idempotent
    }

    if node.Status != StatusRunning && node.Status !=
        StatusReady {
        return fmt.Errorf("INVALID_TRANSITION: cannot mark %s as
        done from status %s", id, node.Status)
    }

    node.Status = StatusDone

    // Decrement in-degree of dependents
    for _, dependent := range d.Adjacency[id] {
        d.InDegree[dependent]--
    }

    d.rebuildReadyQueue()
    return nil
}

// MarkFailed marks a node as failed, blocking its dependents.
func (d *DAG) MarkFailed(id string, reason string) error {
    d.mu.Lock()
    defer d.mu.Unlock()
```

```
    node, ok := d.Nodes[id]
    if !ok {
        return fmt.Errorf("NODE_NOT_FOUND: %s", id)
    }

    node.Status = StatusFailed

    // Cascade block to all transitive dependents
    for _, dependent := range d.Adjacency[id] {
        d.cascadeBlock(dependent)
    }

    d.rebuildReadyQueue()
    return nil
}

func (d *DAG) cascadeBlock(id string) {
    node := d.Nodes[id]
    if node.Status == StatusDone || node.Status == StatusFailed {
        return
    }
    node.Status = StatusBlocked
    for _, dependent := range d.Adjacency[id] {
        d.cascadeBlock(dependent)
    }
}

// MarkRunning marks a node as currently executing.
func (d *DAG) MarkRunning(id string) error {
    d.mu.Lock()
    defer d.mu.Unlock()

    node, ok := d.Nodes[id]
    if !ok {
        return fmt.Errorf("NODE_NOT_FOUND: %s", id)
    }

    if node.Status != StatusReady {
        return
        fmt.Errorf("INVALID_TRANSITION: cannot run %s from
        status %s (must be ready)", id, node.Status)
    }

    node.Status = StatusRunning
    d.rebuildReadyQueue()
    return nil
}
```

```
// IsComplete returns true when all nodes are Done or Failed.
func (d *DAG) IsComplete() bool {
    d.mu.RLock()
    defer d.mu.RUnlock()

    for _, node := range d.Nodes {
        if node.Status != StatusDone && node.Status !=
        StatusFailed {
            return false
        }
    }
    return true
}

// Progress returns (done, total, failed) counts.
func (d *DAG) Progress() (done, total, failed int) {
    d.mu.RLock()
    defer d.mu.RUnlock()

    total = len(d.Nodes)
    for _, node := range d.Nodes {
        switch node.Status {
        case StatusDone:
            done++
        case StatusFailed:
            failed++
        }
    }
    return
}

// Layers returns nodes grouped by their layer (0-indexed).
func (d *DAG) Layers() map[int][]string {
    d.mu.RLock()
    defer d.mu.RUnlock()

    layers := make(map[int][]string)
    for id, node := range d.Nodes {
        layers[node.Layer] = append(layers[node.Layer], id)
    }
    for layer := range layers {
        sort.Strings(layers[layer])
    }
    return layers
}

// ValidateSchema validates that every node conforms to the nano-
        task JSON schema.
```

```go
func (d *DAG) ValidateSchema() []error {
    d.mu.RLock()
    defer d.mu.RUnlock()

    var errs []error
    for id, node := range d.Nodes {
        if id == "" {
            errs = append(errs, errors.New("node has empty ID"))
        }
        if node.Title == "" {
            errs = append(errs, fmt.Errorf("node %s has empty
        title", id))
        }
        if node.Layer < 0 {
            errs = append(errs, fmt.Errorf("node %s has negative
        layer %d", id, node.Layer))
        }
    }
    return errs
}

// rebuildReadyQueue recomputes nodes with in-degree == 0 and
        status == Pending.
// Caller must hold d.mu.
func (d *DAG) rebuildReadyQueue() {
    d.ReadyQueue = nil
    for id, node := range d.Nodes {
        if d.InDegree[id] == 0 && node.Status == StatusPending {
            d.ReadyQueue = append(d.ReadyQueue, id)
            node.Status = StatusReady
        }
    }
    sort.Strings(d.ReadyQueue)
}
```

J. Constitution-Aware Commands — Bash Scripts

File: `scripts/tdd_cycle.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail

# tdd_cycle.sh — Run a complete TDD cycle for a nano-task
# §11.4.224: TDD for ALL work. §11.4.115: RED on broken artifact.
#
# Usage: bash scripts/tdd_cycle.sh <item_id> <test_file>
        <impl_file> [--language=go|ts|bash]
```

```
# Environment:
#   RED_MODE=1 (default) — runs RED phase first
#   TDD_ITERATIONS=3     — deterministic consistency iterations
#   TDD_VERDICT_DIR      — where to write verdict JSONs
#   TDD_EVIDENCE_DIR     — where to write captured evidence

ITEM_ID="${1:?Usage: tdd_cycle.sh <item_id> <test_file>
        <impl_file>}"
TEST_FILE="${2:?}"
IMPL_FILE="${3:?}"
LANG="${4:-go}"
RED_MODE="${RED_MODE:-1}"
ITERATIONS="${TDD_ITERATIONS:-3}"
VERDICT_DIR="${TDD_VERDICT_DIR:-qa-results/tdd_verdicts}"
EVIDENCE_DIR="${TDD_EVIDENCE_DIR:-qa-results/tdd_evidence}"

# Strip --language= prefix if present
LANG="${LANG#--language=}"

mkdir -p "$VERDICT_DIR" "$EVIDENCE_DIR/$ITEM_ID"

# ─── Helpers ──────────────────────────────────────────────────────

sha256_of() {
  # Compute fingerprint of item + files
  local content="${ITEM_ID}"
  for f in "$@"; do
    if [[ -f "$f" ]]; then
      content+=$(<"$f")
    fi
  done
  echo -n "$content" | sha256sum | cut -d' ' -f1
}

now_iso() {
  date -u +"%Y-%m-%dT%H:%M:%SZ"
}

run_test() {
  local mode="$1"
  local lang="$2"
  local test_file="$3"

  case "$lang" in
    go)
      RED_MODE="$mode" go test -v -count=1 "$test_file" 2>&1 ||
        true
```

```
      ;;
    ts|typescript)
      RED_MODE="$mode" npx vitest run "$test_file" --
        reporter=verbose 2>&1 || true
      ;;
    bash|sh)
      RED_MODE="$mode" bash -euo pipefail "$test_file" 2>&1 ||
        true
      ;;
    *)
      echo "UNSUPPORTED_LANGUAGE: $lang" >&2
      return 99
      ;;
  esac
}

# ─── RED Phase ──────────────────────────────────────────────────

run_red() {
  echo "=== RED PHASE: $ITEM_ID ==="

  if [[ ! -f "$TEST_FILE" ]]; then
    echo "RED_FAILED: Test file $TEST_FILE does not exist. Write
        the test first."
    return 1
  fi

  if [[ -f "$IMPL_FILE" ]]; then
    echo "RED_FAILED: Implementation file $IMPL_FILE already
        exists. Delete it and write the test first."
    return 1
  fi

  local red_output
  red_output=$(run_test "1" "$LANG" "$TEST_FILE")
  local red_rc=$?

  if [[ $red_rc -eq 0 ]]; then
    echo "RED_FAILED: Test passed in RED mode — tautology. The
        test does not catch the gap."
    return 1
  fi

  # Deterministic consistency: run N iterations
  local exit_codes=()
  for ((i=1; i<=ITERATIONS; i++)); do
    set +e
```

```
    run_test "1" "$LANG" "$TEST_FILE" > /dev/null 2>&1
    exit_codes+=($?)
    set -e
  done

  local unique_codes
  unique_codes=$(printf '%s\n' "${exit_codes[@]}" | sort -u)
  if [[ $(echo "$unique_codes" | wc -l) -ne 1 ]]; then
    echo "RED_FAILED: Nondeterministic: exit codes across
        $ITERATIONS iterations: ${exit_codes[*]}"
    return 1
  fi

  # Write evidence
  local evidence_file="$EVIDENCE_DIR/$ITEM_ID/RED_${ITEM_ID}
        _stdout.txt"
  echo "$red_output" > "$evidence_file"

  # Write RED verdict
  local fingerprint
  fingerprint=$(sha256_of "$TEST_FILE")

  local verdict_file="$VERDICT_DIR/RED_${ITEM_ID}.json"
  cat > "$verdict_file" <<JSONEOF
{
  "item_id": "$ITEM_ID",
  "guard_identity": "tdd_gate_${ITEM_ID}",
  "polarity": "RED",
  "exit_code": $red_rc,
  "target_fingerprint": "$fingerprint",
  "iterations": $ITERATIONS,
  "evidence_class": "source",
  "evidence_files": ["$evidence_file"],
  "precondition_provenance": "observed",
  "timestamp": "$(now_iso)"
}
JSONEOF

  echo "RED_PASS: Verdict written to $verdict_file"
  echo "RED_PASS: Evidence written to $evidence_file"
  return 0
}

# ─── GREEN Phase
        ──────────────────────────────────────────────────────────

run_green() {
  echo "=== GREEN PHASE: $ITEM_ID ==="
```

```
local red_verdict="$VERDICT_DIR/RED_${ITEM_ID}.json"
if [[ ! -f "$red_verdict" ]]; then
  echo "GREEN_REJECTED: No prior RED verdict at $red_verdict.
      Run RED phase first."
  return 1
fi

local red_fingerprint
red_fingerprint=$(jq -r '.target_fingerprint' "$red_verdict")

if [[ ! -f "$IMPL_FILE" ]]; then
  echo "GREEN_REJECTED: Implementation file
      $IMPL_FILE does not exist."
  return 1
fi

local green_output
green_output=$(run_test "0" "$LANG" "$TEST_FILE")
local green_rc=$?

if [[ $green_rc -ne 0 ]]; then
  echo "GREEN_FAILED: Test did not pass (exit $green_rc)."
  return 1
fi

local green_fingerprint
green_fingerprint=$(sha256_of "$TEST_FILE" "$IMPL_FILE")

if [[ "$green_fingerprint" == "$red_fingerprint" ]]; then
  echo
      "GREEN_FAILED: Identical fingerprints — the artifact has
      not changed since RED. Fix not deployed."
  return 1
fi

# Deterministic consistency
local exit_codes=()
for ((i=1; i<=ITERATIONS; i++)); do
  set +e
  run_test "0" "$LANG" "$TEST_FILE" > /dev/null 2>&1
  exit_codes+=($?)
  set -e
done

local unique_codes
unique_codes=$(printf '%s\n' "${exit_codes[@]}" | sort -u)
if [[ "$unique_codes" != "0" ]]; then
```

```
    echo "GREEN_FAILED: Nondeterministic: not all iterations
        passed (exit codes: ${exit_codes[*]})"
    return 1
  fi

  # Write evidence
  local evidence_file="$EVIDENCE_DIR/$ITEM_ID/GREEN_${ITEM_ID}
        _stdout.txt"
  echo "$green_output" > "$evidence_file"

  # Write GREEN verdict
  local verdict_file="$VERDICT_DIR/GREEN_${ITEM_ID}.json"
  cat > "$verdict_file" <<JSONEOF
{
  "item_id": "$ITEM_ID",
  "guard_identity": "tdd_gate_${ITEM_ID}",
  "polarity": "GREEN",
  "exit_code": 0,
  "target_fingerprint": "$green_fingerprint",
  "iterations": $ITERATIONS,
  "evidence_class": "source",
  "evidence_files": ["$evidence_file"],
  "prior_red_verdict": "$red_verdict",
  "timestamp": "$(now_iso)"
}
JSONEOF

  echo "GREEN_PASS: Verdict written to $verdict_file"
  return 0
}

# ─── Main ──────────────────────────────────────────────────────────

if [[ "$RED_MODE" == "1" ]]; then
  run_red || exit 1
fi

run_green || exit 1

echo ""
echo "=============================================="
echo "TDD CYCLE COMPLETE: $ITEM_ID"
echo "  RED verdict:  $VERDICT_DIR/RED_${ITEM_ID}.json"
echo "  GREEN verdict: $VERDICT_DIR/GREEN_${ITEM_ID}.json"
echo "  Evidence:      $EVIDENCE_DIR/$ITEM_ID/"
echo "=============================================="
```

K. Pack Distribution Script — `pack.sh`

File: `scripts/pack.sh`

```
#!/usr/bin/env bash
set -euo pipefail

# pack.sh — Package the SpecKit-Superpowers bridge for
        distribution
# Produces a self-contained tarball with all source, configs, and
        docs.
#
# §11.4.65: Universal Markdown export — all docs get .html+.pdf
        twins.
# §11.4.73: Project-styling mandate.

VERSION="${1:?Usage: pack.sh <version> (e.g., 1.0.0)}"
OUTPUT_DIR="dist"
PACK_NAME="speckit-superpowers-bridge-${VERSION}"
PACK_DIR="${OUTPUT_DIR}/${PACK_NAME}"

echo "Packing SpecKit-Superpowers Bridge v${VERSION}..."

# Clean previous build
rm -rf "${OUTPUT_DIR:?}/${PACK_NAME:?}"

# Create directory structure
mkdir -p "$PACK_DIR"/
        {src,cmd,pkg,scripts,schemas,config,docs,certs,containers}

# Copy source files
cp src/index.ts "$PACK_DIR/src/"
cp src/granulator.ts "$PACK_DIR/src/"
cp src/implementation-gate.ts "$PACK_DIR/src/"

# Copy Go source
cp -r cmd/ "$PACK_DIR/"
cp -r pkg/ "$PACK_DIR/"

# Copy scripts
cp scripts/tdd_cycle.sh "$PACK_DIR/scripts/"
cp scripts/pack.sh "$PACK_DIR/scripts/"
chmod +x "$PACK_DIR/scripts/"*.sh

# Copy schemas
cp schemas/nano-task.schema.json "$PACK_DIR/schemas/"

# Copy configs
```

```
cp mcp.json "$PACK_DIR/config/"
cp extension.yml "$PACK_DIR/config/"
cp .env.template "$PACK_DIR/config/"

# Copy documentation

# Generate HTML + PDF twins for all Markdown docs
echo "Generating documentation exports..."
for md_file in TDD_INTEGRATION.md SECURITY.md APPENDIX.md; do
  if [[ -f "docs/$md_file" ]]; then
    cp "docs/$md_file" "$PACK_DIR/docs/"

    # Generate HTML
    if command -v pandoc &>/dev/null; then
      pandoc "docs/$md_file" \
        --standalone \
        --from gfm \
        --to html5 \
        --metadata title="$(head -1 "docs/$md_file" | sed 's/
        ^# //')" \
        --output "$PACK_DIR/docs/${md_file%.md}.html" \
        2>/dev/null || echo "WARNING: pandoc failed for $md_file"
    fi

    # Generate PDF
    if command -v pandoc &>/dev/null && command -v weasyprint &>/
        dev/null; then
      pandoc "docs/$md_file" \
        --from gfm \
        --to html5 \
        --metadata title="$(head -1 "docs/$md_file" | sed 's/
        ^# //')" \
        --output /tmp/$$_speckit_doc.html 2>/dev/null && \
      weasyprint /tmp/$$_speckit_doc.html "$PACK_DIR/docs/$
        {md_file%.md}.pdf" 2>/dev/null && \
      rm -f /tmp/$$_speckit_doc.html || \
      echo "WARNING: PDF generation failed for $md_file"
    fi
  fi
done

# Generate DOCX where possible
for md_file in TDD_INTEGRATION.md SECURITY.md APPENDIX.md; do
  if [[ -f "docs/$md_file" ]] && command -v pandoc &>/dev/null;
        then
    pandoc "docs/$md_file" \
      --from gfm \
      --to docx \
```

```
      --output "$PACK_DIR/docs/${md_file%.md}.docx" 2>/dev/null
        || \
      echo "WARNING: DOCX generation failed for $md_file"
  fi
done

# Copy README
if [[ -f README.md ]]; then
  cp README.md "$PACK_DIR/"
fi

# Create tarball
echo "Creating tarball..."
tar -czf "${OUTPUT_DIR}/${PACK_NAME}.tar.gz" -C "$OUTPUT_DIR"
        "$PACK_NAME"

# Create SHA256 checksum
sha256sum "${OUTPUT_DIR}/${PACK_NAME}.tar.gz" > "${OUTPUT_DIR}/$
        {PACK_NAME}.tar.gz.sha256"

echo ""
echo "Pack complete:"
echo "  Tarball: ${OUTPUT_DIR}/${PACK_NAME}.tar.gz"
echo "  SHA256:  $(cat "${OUTPUT_DIR}/$
        {PACK_NAME}.tar.gz.sha256")"
echo "  Size:    $(du -h "${OUTPUT_DIR}/${PACK_NAME}.tar.gz" |
        cut -f1)"
```

L. Environment Template — `.env.template`

```
# ============================================================================
# SpecKit-Superpowers Bridge — Environment Template
# ============================================================================
# Copy this file to .env and fill in the values.
# .env is git-ignored per §11.4.30. NEVER commit real
        credentials.
# ============================================================================

# ─── Helix LLM Gateway
        ──────────────────────────────────────────────────

HELIX_GATEWAY_URL=https://localhost:3101
HELIX_GATEWAY_TLS_CERT=/path/to/certs/gateway-client.crt
HELIX_GATEWAY_TLS_KEY=/path/to/certs/gateway-client.key
HELIX_GATEWAY_CA_CERT=/path/to/certs/ca.crt

# ─── llama.cpp RPC
        ──────────────────────────────────────────────────
```

```
LLAMA_RPC_HOST=localhost
LLAMA_RPC_PORT=8080
LLAMA_RPC_TLS_ENABLED=false
LLAMA_MODEL_PATH=/models/llama-3-8b-instruct.Q4_K_M.gguf
LLAMA_MODEL_SHA256=  # Verified model hash from vendor — fill in

# ─── TDD Enforcement ──────────────────────────────────────────────────────
TDD_VERDICT_DIR=qa-results/tdd_verdicts
TDD_EVIDENCE_DIR=qa-results/tdd_evidence
TDD_ITERATIONS=3
TDD_COVERAGE_THRESHOLD=0.85
TDD_MAX_CYCLES=5
TDD_SANDBOX_ENABLED=true
TDD_TIMEOUT_SECONDS=300

# ─── Container Runtime §11.4.161 ──────────────────────────────────────────
CONTAINER_RUNTIME=podman
CONTAINER_ROOTLESS=true
NANO_TASK_SANDBOX_IMAGE=nano-task-runner:latest
NANO_TASK_MEM_LIMIT=256m
NANO_TASK_CPU_LIMIT=0.5

# ─── External Services ────────────────────────────────────────────────────
# Cloud LLM provider API keys (only used as last-resort fallback
        per §11.4.196)
CLOUD_LLM_PROVIDER=  # e.g., openai, anthropic — optional
CLOUD_LLM_API_KEY=   # NEVER commit

# ─── Project Identity §11.4.151 ───────────────────────────────────────────
HELIX_RELEASE_PREFIX=speckit-bridge

# ─── ClickUp Integration §11.4.148(D5) ────────────────────────────────────
CLICKUP_API_TOKEN=       # NEVER commit
CLICKUP_LIST_ID=         # Workable items list
CLICKUP_WORKSPACE_ID=    # Workspace ID
CLICKUP_ASSIGNEE=        # Default assignee username

# ─── GitHub/GitLab CLIs ───────────────────────────────────────────────────
GITHUB_TOKEN=    # NEVER commit
GITLAB_TOKEN=    # NEVER commit
```

M. Container Compose Files

File: `containers/docker-compose.yml`

```yaml
# docker-compose.yml — SpecKit-Superpowers Bridge Services
# All services run rootless per §11.4.161.
# Provisioned via vasic-digital/containers submodule §11.4.76.

version: "3.9"

services:
  # ── Helix LLM Gateway ──────────────────────────────────────
  helix-gateway:
    image: "helix-gateway:latest"
    build:
      context: .
      dockerfile: containers/Dockerfile.gateway
    ports:
      - "3101:3101"
    volumes:
      - ./config/gateway.json:/app/config/gateway.json:ro
      - ./certs:/app/certs:ro
      - ./models:/models:ro
    environment:
      - GATEWAY_CONFIG=/app/config/gateway.json
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    user: "1000:1000"
    mem_limit: "512m"
    cpus: "1.0"

  # ── llama.cpp Inference Server ─────────────────────────────
  llama-server:
    image: "llama-cpp-server:latest"
    build:
      context: .
      dockerfile: containers/Dockerfile.llama
    ports:
      - "8080:8080"
    volumes:
      - ./models:/models:ro
    environment:
      - LLAMA_MODEL=/models/llama-3-8b-instruct.Q4_K_M.gguf
```

```
      - LLAMA_HOST=0.0.0.0
      - LLAMA_PORT=8080
      - LLAMA_N_GPU_LAYERS=35
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    user: "1000:1000"
    mem_limit: "24g"

  # ─── Audit Log Aggregator ──────────────────────────────────
  audit-logger:
    image: "audit-logger:latest"
    build:
      context: .
      dockerfile: containers/Dockerfile.audit
    volumes:
      - ./qa-results/audit:/var/log/audit
    environment:
      - AUDIT_LOG_DIR=/var/log/audit
      - AUDIT_RETENTION_DAYS=90
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    user: "1000:1000"
    mem_limit: "128m"
```

File: containers/Dockerfile.gateway

```
# Dockerfile.gateway — Helix LLM Gateway build
FROM golang:1.22-alpine AS builder

WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY cmd/ ./cmd/
COPY pkg/ ./pkg/
```

```
RUN CGO_ENABLED=0 GOOS=linux go build -o /gateway ./cmd/gateway

FROM alpine:3.20
RUN apk add --no-cache ca-certificates tzdata
COPY --from=builder /gateway /usr/local/bin/gateway
USER 1000:1000
EXPOSE 3101
ENTRYPOINT ["/usr/local/bin/gateway"]
CMD ["--config", "/app/config/gateway.json"]
```

N. Test Artifact Template

File: `templates/nano_task_test_template.go.tmpl`

```
package nano_task

import (
    "os"
    "testing"
)

// TestNanoTask_{{.ItemID}} is the TDD guard for nano-task
        {{.ItemID}}.
// Generated from speckit-superpowers-bridge template.
// RED_MODE=1: assert the defect/gap is present (no
        implementation yet).
// RED_MODE=0: assert the implementation is correct (§11.4.115
        polarity switch).

func TestNanoTask_{{.ItemID}}(t *testing.T) {
    redMode := os.Getenv("RED_MODE")
    if redMode == "" {
        redMode = "1"
    }

    // Precondition: implementation is absent or broken in RED
        mode
    // The test must genuinely catch the gap — not be a
        tautology.

    // Arrange
    input := "{{.TestInput}}"
    expected := "{{.ExpectedOutput}}"

    // Act
    result, err := {{.FunctionName}}(input)
```

```
    // Assert — RED mode
    if redMode == "1" {
        if err == nil && result == expected {
            t.Fatalf("RED_TAUTOLOGY: {{.FunctionName}} returned
        correct result in RED mode. " +
                "The test is not catching the gap. Is the
        implementation already present?")
        }
        t.Logf("RED_PASS: {{.FunctionName}} correctly reports
        broken/absent in RED mode (err=%v)", err)
        return
    }

    // Assert — GREEN mode
    if err != nil {

        t.Fatalf("GREEN_FAILED: {{.FunctionName}} returned
        error: %v", err)
    }
    if result != expected {
        t.Fatalf("GREEN_FAILED: {{.FunctionName}}(%q) = %q, want
        %q", input, result, expected)
    }
    t.Logf("GREEN_PASS: {{.FunctionName}}(%q) = %q", input,
        result)
}
```

O. CI Configuration — Disabled per §11.4.156

File: `.github/workflows/constitution-compliance.yml.disabled-local-only`

```
# ============================================================================
# DISABLED — per §11.4.156: All CI/CD automation MUST be
        disabled.
# ============================================================================
# This file is deliberately named .disabled-local-only so GitHub
        Actions
# does NOT execute it. Remote CI is DISABLED; enforcement
        migrates to
# LOCAL git hooks (§11.4.75) + pre-build verification (§11.4.40).
#
# NEVER rename this to .yml — that would re-enable remote CI,
        which is a
# §11.4.156 violation and a release blocker.
# ============================================================================
#
```

```
# What this workflow DID before being disabled (preserved for
        reference):
#
# name: Constitution Compliance
# on:
#   push:
#     branches: [main]
#   pull_request:
#     branches: [main]
#
# jobs:
#   pre-build-verification:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#         with:
#           submodules: recursive
#       - name: Run pre-build gates
#         run: bash constitution/scripts/verify-all-constitution-
        rules.sh
#
#   meta-test:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#         with:
#           submodules: recursive
#       - name: Run meta-test mutations
#         run: bash scripts/testing/
        meta_test_false_positive_proof.sh
#
#   propagation-gates:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#         with:
#           submodules: recursive
#       - name: Verify propagation
#         run: bash constitution/scripts/gates/CM-COVENANT-*.sh
#
# ============================================================================
# All CI jobs are now LOCAL-ONLY per §11.4.156 + §11.4.75 Layer
        5.
# ============================================================================
```

End of Appendix.